Senior SRE & Cloud Leadership

Interview preparation playbook | GitHub Actions • Git • Docker • Kubernetes/EKS • AWS • Reliability • Role transitions

Prepared for Nandakumar Athiseshan | October 2026

Purpose: A single practice document combining a five-minute career story, tailored role-transition narratives, Cisco security-role analysis, 30 technical interview questions with GitHub Actions (not Jenkins), and senior-level interview coaching.

How to use this guide

- Practice the five-minute story aloud; do not read it verbatim.
- Answer technical questions in this order: impact → diagnosis/design → commands or implementation → verification → prevention.
- Use real evidence from your resume. Do not claim direct Cisco ISE, TrustSec, CCIE or Product Manager ownership unless you can substantiate it.

PART I — Five-minute career introduction

Opening (30 seconds)

Thank you for the opportunity. I have more than 14 years of experience at Cisco, spanning customer support, production operations, SRE, AWS cloud infrastructure and platform engineering. My career has progressed from resolving critical customer-impacting problems to designing reusable cloud platforms and leading complex production migrations.

Operations foundation (45 seconds)

I started in customer support and release operations. That gave me first-hand experience with SLAs, incident response, observability and root-cause analysis. It taught me to think from the customer impact backwards, and to design systems that are operable—not just deployable.

DevOps and cloud transition (45 seconds)

I moved into CI/CD automation, Kubernetes and AWS. I worked with Terraform and Terragrunt, Git-based delivery, Argo CD and enterprise monitoring. My focus was removing manual steps and enabling repeatable, secure releases while improving reliability.

Platform architecture (45 seconds)

At Cisco IQ, I helped build repeatable AWS foundations spanning 15 AWS accounts and four regions. The work included multi-account identity, networking, DNS, certificates, load balancing and infrastructure-as-code modules. I collaborated across application, network, security and platform teams to make onboarding more predictable.

Current migration, troubleshooting and automation (90 seconds)

In my current Software Engineering Technical Leader role, I lead SRE workstreams for moving CX AI Assistant and related AI/ML applications from on-premises OpenShift to AWS EKS across development, nonproduction and production. I work on readiness, domain delegation, ACM certificate lifecycle, connectivity, secrets and GitOps

cutovers. In one incident, diagnosing a missing security-group ingress rule restored a nonproduction EKS cluster to 36 healthy pods across 11 nodes. I also contributed AI-assisted Jira-to-PR workflows that reduced PR-authoring cycle time by about 60%.

Close (45 seconds)

My strength is combining hands-on production depth with cross-functional delivery. I understand how to build cloud platforms, but also how to protect their reliability, manage change and explain technical trade-offs. I am interested in a role where I can increase that impact through platform strategy, technical consulting, security architecture or product-led engineering.

Speaking plan

Time	Topic
0:00–0:30	Who you are
0:30–1:15	Support/operations foundation
1:15–2:00	DevOps/SRE transition
2:00–2:45	Platform architecture
2:45–4:15	Current migration, incident and automation
4:15–5:00	Strengths and future direction

PART II — Repositioning your background for new roles

Core rule: move from the implementation ("I configured Route 53") to the stakeholder problem, decision, delivery accountability and impact ("I enabled secure multi-account service onboarding and reliable production cutovers").

Engineering description	Leadership / product framing
Migrated OpenShift workloads to EKS	Led modernization workstreams and coordinated production-readiness risks
Configured IAM, DNS, ACM and VPC components	Delivered secure connectivity, identity and service-access capabilities
Built reusable Terraform modules	Standardized a platform capability used across accounts and regions
Fixed a production incident	Directed evidence-driven diagnosis, service restoration and prevention
Automated Jira-to-PR creation	Identified repetitive work and delivered measurable workflow improvement

A. Security consulting / technical leadership introduction

I bring 14+ years at Cisco across enterprise customer support, production operations and cloud platform leadership. My current work combines AWS, Kubernetes, network connectivity, identity and access management, secrets, certificate lifecycle and complex migrations. I help teams translate requirements into secure production-ready infrastructure, align dependencies across security, network and application organizations, and resolve critical reliability issues. I'm interested in extending that experience into enterprise technical consulting and security architecture. I would be transparent that my documented strength is cloud/platform security; I would separately detail any hands-on ISE or TrustSec experience if the position requires it.

B. Technical Product Manager introduction

I have spent 14+ years solving customer and engineering problems, from production support to cloud platform architecture. Over time, I've increasingly worked on understanding internal developer needs, prioritizing onboarding and reliability improvements, standardizing reusable capabilities and coordinating stakeholders. Our multi-account AWS foundations and Jira-to-PR automation are examples of translating technical friction into a better platform experience, including about 60% faster PR authoring. I want to bring this technical foundation into product management with more explicit ownership of user discovery, prioritization, roadmap decisions, adoption and business outcomes.

C. Why are you changing roles?

For a Product Manager role

I enjoy building technology, but the part of my work I increasingly value is identifying the right problem, aligning stakeholders and measuring whether a solution helped users. I want to move from primarily implementing platforms to helping own what to build, why it matters and how to measure adoption and impact.

For security consulting / architect leadership

My work has evolved toward cross-functional architecture, production-risk management and secure cloud delivery. A consulting role would let me apply that experience closer to enterprise customer requirements and end-to-end solution delivery. I see this as an expansion of my technical foundation, not a departure from it.

D. Cisco security job fit: credible positioning

Requirement	Positioning
10+ years enterprise IT	Strong match: 14+ years at Cisco
Cloud security / enterprise networking	Strong relevant cloud IAM, SG/NACL, VPC, DNS, TLS and certificate experience
Cisco ISE design/implementation	Not evidenced in current resume; do not imply direct delivery
TrustSec production rollout	Not evidenced in current resume; do not imply ownership
Multi-vendor security troubleshooting	Present adjacent troubleshooting evidence; provide specific cases if available
CCIE Security	Not listed; alternative certifications include AWS SA-Pro, CKA, CKAD, Terraform
Cross-functional consulting/leadership	Strong examples from migrations, platform standards and cutovers

Interview bridge: "My deepest expertise is enterprise cloud and platform security. I understand how identity, policy, segmentation and connectivity affect production delivery. I would not overstate hands-on ISE/TrustSec deployment history; I can describe exactly where my experience overlaps and how I would close the platform-specific gap."

PART III — Technical questions and interview-ready answers

A. GitHub Actions & CI/CD (replaces Jenkins entirely)

1. How do you design an end-to-end CI/CD pipeline with GitHub Actions?

I use pull-request workflows for linting, unit tests, SAST and quality checks, dependency scanning and image validation. After an approved merge to a protected branch, a release workflow builds one immutable image, scans it, publishes it to ECR using OIDC-based AWS credentials and records its digest. For EKS I prefer GitOps: update the deployment manifest in a controlled repo, let Argo CD reconcile it, then verify rollout health and rollback criteria. Dev and non-prod promotion should be automated; production requires policy-based review, monitoring and auditable approval.

2. How do you securely manage secrets and AWS access in GitHub Actions?

Prefer GitHub OIDC federation to assume a narrowly scoped AWS IAM role, rather than long-lived AWS keys. Store other required secrets in GitHub Actions environment/repository/organization secrets or a cloud secret manager, restrict access by environment and reviewers, avoid printing credentials and use least-privilege GITHUB_TOKEN permissions. Pin trusted third-party actions to commit SHAs. Secrets may not be available to untrusted fork pull requests by design.

3. How do you troubleshoot a failed GitHub Actions workflow?

Locate the failing job and step in the run logs. Check the event and branch/ref, runner OS and permissions, changed dependencies, caching, artifact paths and action versions. For AWS/ECR failures, inspect OIDC trust conditions, role permissions and region settings. For EKS failures, inspect GitOps sync, deployment events, pod logs and image digests. Re-run only after understanding whether the step is idempotent; do not blindly retry production deployment.

4. What are workflows, jobs, steps, runners and reusable workflows?

A workflow is YAML in .github/workflows triggered by events or schedules. A job runs on a runner and contains steps; jobs can depend on other jobs using needs. Steps execute shell commands or actions. Reusable workflows, called with workflow_call, let teams share standardized CI/CD pipelines across repositories; composite actions package reusable steps.

5. How do you require approval before production deployment in GitHub Actions?

Create a protected production environment with required reviewers, allowed deployment branches or tags, and optionally wait timers. Reference environment: production in the deployment job so GitHub enforces its protection rules before releasing environment secrets and executing the job. Restrict production cloud role trust to the intended repository, branch and environment. Availability of specific protection features can depend on the GitHub plan and repository visibility.

6. How do you integrate SonarQube and Trivy with GitHub Actions?

Run SonarQube scanning and enforce a project quality gate as part of PR or release checks. Build an image, scan it with Trivy for vulnerabilities and optionally configuration/IaC issues, and fail based on defined severity thresholds. Pin tool versions, upload SARIF where useful, document exception policy, and ensure critical findings block promotion rather than only being reported.

7. GitHub Actions vs Jenkins: why choose GitHub Actions?

GitHub Actions offers native pull-request integration, hosted or self-hosted runners, reusable workflows, environments and OIDC federation. Jenkins can provide greater customization for complex legacy environments but needs separate controller/plugin/agent management. I select based on scale, governance, network access, operational overhead and required integrations; I do not frame one as universally better.

8. Show a secure GitHub Actions ECR deployment pattern

Use permissions: id-token: write and contents: read; aws-actions/configure-aws-credentials to assume a scoped AWS role; aws-actions/amazon-ecr-login; docker build/tag/push; and capture the pushed image digest. Protect deployment environments and promote that digest through GitOps. Avoid exposing credentials to untrusted PR code.

Illustrative YAML skeleton (replace placeholders and pin tested action commit SHAs in production):

```
name: release
on:
  push:
    branches: [main]
permissions:
  contents: read
  id-token: write
jobs:
  publish:
    runs-on: ubuntu-latest
    environment: nonprod
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ecr-publisher
          aws-region: us-west-2
      - uses: aws-actions/amazon-ecr-login@v2
      - run: |
          docker build -t myapp:${{ github.sha }} .
          docker tag myapp:${{ github.sha }} 123456789012.dkr.ecr.us-west-2.amazonaws.com/myapp:${{ github.sha }}
          docker push 123456789012.dkr.ecr.us-west-2.amazonaws.com/myapp:${{ github.sha }}
```

B. Git & GitHub

9. What is git merge vs git rebase?

Merge combines histories and can preserve a branch merge commit; rebase replays commits on a new base and rewrites commit IDs. Rebase local short-lived branches for clean review when appropriate, but avoid rewriting public shared branch history. Use merge where preserving original topology matters.

```
git fetch origin && git rebase origin/main
```

10. How do you resolve merge conflicts?

Identify conflicted files with git status, inspect both sides, edit according to the desired behavior, then test the application. Stage resolved files and continue the merge or rebase. Do not accept one side indiscriminately.

```
git status  # edit conflicts
git add <file>
git rebase --continue  # or git commit for a merge
```

11. How do you protect sensitive information from being pushed?

Use .gitignore for untracked local config, pre-commit scanners, GitHub secret scanning and push protection, branch protections and reviews. Keep deployment secrets in approved secret managers and prefer workload identity. If exposed, revoke/rotate immediately and assess access; deleting the line or commit does not erase the exposure.

12. How do you implement a branching strategy?

For frequent deployment use trunk-based development: short-lived feature branches, reviewed PRs, status checks, protected main and immutable release tags. Use release branches only when supporting parallel versions; keep production promotion separate from ordinary feature branching.

C. Docker

13. Image vs container?

An image is the packaged filesystem/configuration template; a container is an instantiated runtime process plus a writable layer and isolation settings. Multiple containers can run from one image.

14. Container repeatedly restarts: how do you troubleshoot?

Inspect logs, exit codes, restart count, startup command, health checks, CPU/memory and dependencies. Exit code 137 may be a SIGKILL/OOM; verify the actual termination reason. On EKS, describe the pod, review previous logs and Kubernetes events.

```
docker ps -a; docker logs --tail 100 <id>; docker inspect <id>
```

15. How do you optimize a Dockerfile?

Use trusted minimal bases, multi-stage builds when useful, pinned dependencies, .dockerignore, layer-aware COPY ordering, build caching and a non-root user. Keep build-time secrets out of layers, scan the final image and prefer digest-pinned base images for repeatability.

16. How do you push Docker images to Amazon ECR?

Create/confirm the ECR repository, authenticate with aws ecr get-login-password, build, tag and push. In CI, prefer GitHub Actions OIDC to get temporary AWS role credentials and promote an image digest rather than mutable latest tags.

```
aws ecr get-login-password --region us-west-2 | docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-west-2.amazonaws.com
docker build -t myapp:1.0 .
docker tag myapp:1.0 123456789012.dkr.ecr.us-west-2.amazonaws.com/myapp:1.0
docker push 123456789012.dkr.ecr.us-west-2.amazonaws.com/myapp:1.0
```

D. Kubernetes / EKS

17. CrashLoopBackOff vs ImagePullBackOff?

CrashLoopBackOff indicates repeated start/crash cycles: inspect previous logs, probes, configuration and OOM kills. ImagePullBackOff means image retrieval failed: check image name/digest, repository, IAM authorization, registry reachability and ECR endpoints.

```
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
```

18. What happens when a worker node fails?

The node becomes NotReady/Unknown when health monitoring detects failure. After toleration/eviction handling, controllers can replace eligible pods on other nodes if capacity, storage and constraints permit. Check EKS managed node groups, autoscaling/Karpenter, nodes, PDBs and available replicas.

```
kubectl get nodes -o wide; kubectl get pods -A -o wide
```

19. Deployment vs StatefulSet vs DaemonSet?

Deployment: replaceable stateless replicas and rolling updates. StatefulSet: stable identity and optionally per-pod persistent storage, useful for stateful clusters. DaemonSet: a copy on each eligible node, such as log collectors or agents.

20. Readiness vs liveness vs startup probes?

Readiness determines inclusion in Service endpoints. Liveness failure restarts a container; startup probe gives slow-starting services time before other probes apply. Avoid designing probes that cause cascading restarts due solely to temporary downstream outages.

21. How do you perform near-zero-downtime deployments?

Use multiple healthy replicas, RollingUpdate maxUnavailable: 0, readiness, graceful shutdown and connection draining, capacity headroom, backward-compatible schema changes, health/latency checks and rollback. Verify ingress/ALB target propagation; Kubernetes rolling update by itself is not a guarantee.

```
kubectl rollout status deployment/<name> -n <ns>
kubectl rollout undo deployment/<name> -n <ns>
```

22. Pod Pending: what do you check?

Inspect pod scheduling events: resource requests, node affinity, taints/tolerations, PVC binding, topology, quotas and node limits. In EKS also inspect subnet IP availability and Karpenter/Cluster Autoscaler provisioning and EC2 capacity.

```
kubectl describe pod <pod> -n <ns>
kubectl get pvc -n <ns>
kubectl describe nodes
```

E. AWS

23. Explain VPC, subnet, route table and security groups.

A VPC isolates an AWS network. A subnet is an AZ-scoped IP range within it. Route tables determine next hops for destinations. Security groups are stateful allow-rule firewalls on interfaces/resources, while subnet NACLs are stateless network filters. Subnet “public” status depends on routing and public addressing—not only its name.

24. ALB vs NLB vs CloudFront?

ALB is HTTP/HTTPS-aware Layer 7 routing by host/path and integrates with application targets. NLB handles transport-layer TCP/UDP/TLS load balancing. CloudFront is an edge content distribution/proxy service for caching, acceleration and edge protections; it can sit in front of an ALB origin.

25. How do you secure an AWS environment?

Use organization/account separation, SCPs where applicable, least-privilege IAM and MFA, short-lived role credentials, segmentation/egress controls, TLS and KMS encryption, Secrets Manager, CloudTrail and security findings, workload patching and scans, compliance-as-code and documented incident handling. Validate controls in CI and continuously.

26. EC2 unreachable: how do you troubleshoot?

Establish whether failure is instance health, network path, authentication or process. Check instance/status checks, public/private IP and DNS, security groups, NACLs, subnet routes, IGW/VPN/TGW, host firewall and service listener. Use VPC Reachability Analyzer, Flow Logs and SSM Session Manager when permitted.

27. How do you monitor applications with CloudWatch?

Collect EC2/EKS/ALB metrics, app logs and traces; use CloudWatch Agent or Container Insights for memory/disk and container visibility. Build dashboards and alarms for request error rate, p95/p99 latency, saturation and downstream errors; route actionable alerts to on-call and measure SLOs.

F. SRE / Troubleshooting

28. CPU suddenly reaches 95–100%: what do you do?

First identify customer impact and whether usage is node, pod or process-specific. Compare traffic, recent deployments, CPU throttling and latency; look for hot loops, large queries or jobs. Mitigate with safe scaling or rollback if needed, confirm recovery, then profile and address root cause.

```
top; ps aux --sort=-%cpu | head
kubectl top pods -n <ns>; kubectl top nodes
```

29. How do you troubleshoot HTTP 503?

Determine which layer sent 503: CloudFront, ALB/Ingress, Service or application. Check target registration/health, ready pod endpoints, deployments, readiness, capacity, error logs and dependency failures. Correlate with release time, restore healthy targets and validate recovery from the client path.

```
kubectl get svc,endpoints -n <ns>
kubectl describe ingress <ingress> -n <ns>
```

30. Latency increased from 200 ms to 5 seconds: investigate?

Compare p50/p95/p99, traffic and error rates across regions/endpoints. Use traces to isolate app execution, slow SQL, queues, downstream RPC and networking. Correlate recent deploys, saturation, connection-pool pressure and cache misses. Mitigate with rollback/scale where justified; verify latency restoration and add targeted alerts.

31. Disk usage reached 95%: next steps?

Identify the filesystem and whether inode exhaustion is involved, measure growth and locate heavy paths. Apply safe log retention/rotation, prune confirmed obsolete artifacts, expand storage where appropriate and avoid deleting active files. Look for deleted-but-open files and put capacity forecasts in place.

```
df -h; df -i; du -xhd1 /var 2>/dev/null; sudo lsof +L1
```

32. How do you monitor availability and latency? What are SLI and SLO?

An SLI is a measured service behavior, such as successful-request ratio or request latency. An SLO is a target for that SLI over a time window, e.g. 99.9% successful valid requests over 30 days or 99% under 300 ms. Monitor user-facing SLIs and burn-rate alerts; an error budget is the permitted unreliability. An SLA is a formal commitment, potentially contractual.

PART IV — Senior-level follow-up scenarios

Pods say Running, but clients see 503

Running does not imply Ready. Follow CloudFront → ALB targets → Ingress → Service endpoints → pod readiness → downstream health. Use target health, events and access logs, then safely restore healthy targets.

New version deployed but clients still see old content

Compare container digests and deployed ReplicaSets, Argo CD sync, selectors, ALB target groups and CloudFront caching. Check split traffic and propagation before changing configuration.

CoreDNS fails only for a subset of pods

Compare healthy and impacted nodes, kube-dns Service endpoints, pod resolver settings, CoreDNS logs, SG/NACL, network policies and DNS query patterns. Test from an affected pod.

AWS cost unexpectedly triples

Identify affected account, service, region and usage type using Cost Explorer/CUR. Investigate node scaling, NAT processing, inter-AZ transfer, log ingestion, duplicate resources and recent deployments.

CI/CD takes 40+ minutes

Profile each job, separate sequential bottlenecks, parallelize independent checks, cache dependencies carefully, use incremental builds and right-size runners without bypassing security gates.

Terraform plan shows unexpected drift

Confirm correct workspace/account/state and provider versions, inspect cloud changes, reconcile approved emergency changes in code, prevent parallel applies and review for potentially destructive actions.

PART V — Leadership and interview technique

How to turn any technical project into a leadership example

- Business problem: Who was blocked, what risk/cost/delay existed?
- Scope and ownership: What workstream and decisions were yours?
- Trade-offs and stakeholders: What alternatives did you evaluate, and who needed to align?
- Outcome: What improved, with numbers you can defend?
- Learning: What standard, monitoring or process reduced recurrence?

Five delivery tips

- Lead with outcomes; use technology as supporting detail.
- Spend most of the five-minute introduction on recent, directly relevant work.
- Keep one real incident (CoreDNS/security group) ready as a two-minute deep dive.
- Be precise about your own role versus the team's accomplishments.
- Close by connecting experience to the specific job rather than reciting certifications.

Likely follow-up questions

- How did you sequence your on-prem OpenShift to EKS production migrations?
- What did you personally own in AWS networking, DNS, ACM and certificate cutovers?
- How did you isolate the CoreDNS/security-group issue?
- How do you organize Terraform modules across 15 accounts and four regions?
- What safeguards make your Jira-to-PR automation safe and auditable?
- Tell me about a disagreement with security or networking and how you resolved it.
- As a Product Manager, how would you prioritize platform features and measure adoption?
- For this security role, what direct ISE and TrustSec implementations have you delivered?

Recommended 60–90 second answer structure

> Situation (10 sec): the business impact and constraints. Responsibility (10 sec): what you owned. Action (35 sec): key decisions, collaboration and technical detail. Result (15 sec): quantified outcome or verified recovery. Learning (10 sec): prevention or repeatable standard.

Final readiness checklist

- Practice a natural 5-minute introduction and a shorter 90-second version.
- Prepare two STAR stories for migrations, one incident, one automation and one cross-team conflict.
- Know GitHub Actions environments, OIDC, permissions and reusable workflows—not Jenkins commands.
- Practice kubectl, AWS network triage, GitOps rollback and ECR release promotion.
- Be factual about Cisco ISE, TrustSec, multi-vendor security and CCIE qualifications.
- Know exactly why the target role is a progression and what outcomes you will own.

Note: The career experience and accomplishments are adapted from the uploaded resume and our earlier interview drafts. Tool descriptions, commands, general technical guidance and role-transition framing are educational examples, not claims that every practice was implemented at Cisco.